

PIPELINE CI/CD COM TESTES AUTOMATIZADOS

API • E2E Web • Mobile

Proposta de configuração com GitHub Actions e modelo de relatório de execução

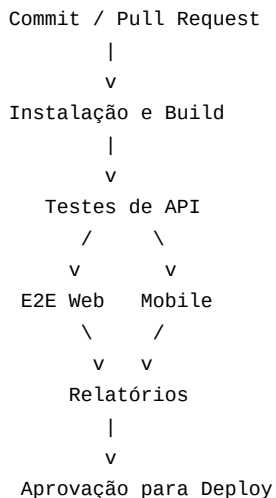
1. Objetivo

O objetivo é configurar um pipeline de Integração Contínua (CI) utilizando GitHub Actions, capaz de executar automaticamente testes de API, E2E Web e Mobile após cada alteração relevante no repositório. O pipeline deve detectar falhas antes que o código seja incorporado à branch principal ou disponibilizado em produção.

A estratégia transforma os testes automatizados em uma etapa obrigatória do processo de desenvolvimento, aumentando a confiabilidade das entregas e reduzindo regressões.

2. Arquitetura do Pipeline

Fluxo proposto:



2.1 Checkout

O GitHub Actions obtém a versão do código correspondente ao commit que iniciou a execução.

2.2 Configuração do ambiente

São instaladas as dependências necessárias para a aplicação e para os testes, como Node.js, npm, Newman/Postman, Playwright, Java, Appium e Android SDK.

2.3 Testes de API

Os testes de API podem ser executados com Postman + Newman, Jest + Supertest ou REST Assured. Nesta proposta é utilizado Newman. A suíte valida status HTTP, autenticação, payloads, estrutura das respostas, regras de negócio, tratamento de erros e, quando aplicável, tempo de resposta.

2.4 Testes E2E Web

A automação E2E utiliza Playwright para simular comportamentos reais do usuário: acesso à aplicação, login, navegação, preenchimento de formulários, execução de operações e validação dos resultados apresentados pela interface.

2.5 Testes Mobile

A automação mobile utiliza Appium com emulador Android. Os testes podem validar inicialização do aplicativo, autenticação, navegação, botões, formulários, fluxos críticos e integração com a API.

2.6 Evidências

Independentemente do resultado, o pipeline deve gerar artefatos como relatórios, logs, screenshots de falhas, traces E2E e resultados JUnit/XML. Isso facilita a investigação de problemas pelo time de desenvolvimento e QA.

3. Configuração do GitHub Actions

O arquivo de workflow pode ser criado em:

```
.github/workflows/automated-tests.yml
```

Exemplo de configuração:

```
name: CI - Automated Tests

on:
  push:
    branches:
      - main
      - develop
  pull_request:
    branches:
      - main
      - develop

jobs:
  api-tests:
    name: API Tests
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install Newman
        run: npm install -g newman newman-reporter-htmlextra

      - name: Run API tests
```

```

run: |
  newman run tests/api/api-tests.postman_collection.json \
  -e tests/api/environment.postman_environment.json \
  -r cli,htmlextra,junit \
  --reporter-htmlextra-export reports/api-report.html \
  --reporter-junit-export reports/api-junit.xml

- name: Upload API report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: api-test-report
    path: reports/

e2e-tests:
  name: E2E Web Tests
  runs-on: ubuntu-latest
  needs: api-tests
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: 20

    - name: Install dependencies
      run: npm ci

    - name: Install Playwright
      run: npx playwright install --with-deps

    - name: Run E2E tests
      run: npx playwright test

    - name: Upload Playwright report
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: e2e-playwright-report
        path: playwright-report/

    - name: Upload screenshots and traces
      if: failure()
      uses: actions/upload-artifact@v4
      with:
        name: e2e-failure-evidence
        path: test-results/

mobile-tests:
  name: Mobile Tests
  runs-on: ubuntu-latest
  needs: api-tests
  steps:
    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Setup Node

```

```

    uses: actions/setup-node@v4
    with:
      node-version: 20

- name: Setup Java
  uses: actions/setup-java@v4
  with:
    distribution: temurin
    java-version: 17

- name: Install dependencies
  run: npm ci

- name: Install Appium
  run: |
    npm install -g appium
    appium driver install uiautomator2

- name: Run Android Emulator and tests
  uses: reactivecircus/android-emulator-runner@v2
  with:
    api-level: 33
    target: google_apis
    arch: x86_64
    script: npm run test:mobile

- name: Upload Mobile report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: mobile-test-report
    path: reports/mobile/

pipeline-result:
  name: Final Validation
  runs-on: ubuntu-latest
  needs:
    - api-tests
    - e2e-tests
    - mobile-tests
  steps:
    - name: Pipeline approved
      run: echo "Todos os testes foram executados com sucesso."

```

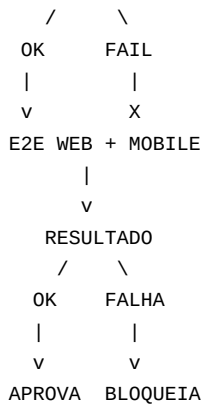
4. Estratégia de Execução

Após um desenvolvedor realizar um commit e enviar as alterações ao repositório, o GitHub Actions inicia automaticamente o pipeline.

```

git commit -m "Nova funcionalidade"
git push
COMMIT / PUSH
|
v
GitHub Actions
|
v
API TESTS

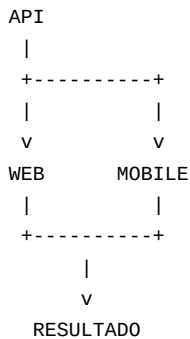
```



Uma falha em uma suíte obrigatória impede que o pipeline seja considerado aprovado, evitando que versões com erros conhecidos avancem no processo de entrega.

5. Execução Paralela

Após a validação da API, as suítes Web e Mobile podem ser executadas em paralelo. Essa abordagem reduz o tempo total da integração contínua e melhora o feedback ao desenvolvedor.



6. Variáveis e Informações Confidenciais

Senhas, tokens e URLs privadas não devem ser inseridos diretamente no YAML. Devem ser armazenados em GitHub Secrets: Settings > Secrets and variables > Actions.

- API_URL
- API_TOKEN
- TEST_USER
- TEST_PASSWORD
- DATABASE_TEST_URL

```

env:
  API_URL: ${ secrets.API_URL }
  API_TOKEN: ${ secrets.API_TOKEN }
  
```

7. Estratégia Recomendada para os Testes

7.1 API

Os testes de API devem ser executados primeiro, por serem normalmente mais rápidos e por validarem a base usada pelas camadas Web e Mobile.

- POST /login
- GET /users
- POST /orders
- PUT /orders/:id
- DELETE /orders/:id

Principais validações: HTTP Status, Response Body, Schema, autenticação, regras de negócio e tratamento de erros.

7.2 E2E Web

Os testes E2E devem proteger os fluxos mais importantes para o usuário, sem transformar a interface em uma suíte excessivamente lenta.

Login -> Dashboard -> Cadastrar pedido -> Finalizar pedido -> Consultar pedido

7.3 Mobile

A suíte mobile deve priorizar os fluxos críticos. Como emuladores consomem mais recursos, recomenda-se executar smoke tests em cada commit e uma suíte completa em execuções programadas ou antes de releases.

Abrir aplicativo -> Login -> Tela inicial -> Executar operação -> Confirmar resultado

8. Critérios para Aprovação

Tipo	Critério
API	100% dos testes críticos aprovados
Web E2E	100% dos fluxos críticos aprovados
Mobile	100% dos smoke tests aprovados
Build	Compilação realizada sem erro
Pipeline	Nenhum job obrigatório apresentar falha

Branches como main podem utilizar Branch Protection Rules para bloquear o merge quando o pipeline estiver com falha.

Teste falhou -> Merge bloqueado -> Correção -> Novo commit -> Nova execução automática

9. Modelo de Relatório de Execução

Os dados abaixo são um modelo de apresentação. Em uma execução real, devem ser substituídos pelos resultados obtidos no GitHub Actions.

9.1 Identificação

Campo	Valor
Pipeline	CI - Automated Tests
Plataforma	GitHub Actions
Trigger	Push
Branch	develop
Commit	exemplo
Ambiente	Homologação

9.2 Resultados - API

Métrica	Resultado
Total de testes	42

Aprovados	42
Falhas	0
Ignorados	0
Taxa de sucesso	100%
Status	APROVADO

- Autenticação
- Consulta de usuário
- Cadastro
- Atualização
- Criação de pedido
- Consulta de pedidos
- Tratamento de requisições inválidas

9.3 Resultados - E2E Web

Métrica	Resultado
Total de testes	18
Aprovados	18
Falhas	0
Taxa de sucesso	100%
Status	APROVADO

- Login
- Dashboard
- Cadastro
- Pesquisa
- Criação de registro
- Alteração
- Logout

Evidências: Playwright HTML Report, screenshots, traces e logs.

9.4 Resultados - Mobile

Métrica	Resultado
Total de testes	12
Aprovados	12
Falhas	0
Taxa de sucesso	100%
Status	APROVADO

- Inicialização do aplicativo
- Login
- Navegação
- Consulta
- Cadastro
- Logout

Ambiente de referência: Android Emulator, Android API 33, Appium e UiAutomator2.

10. Resultado Consolidado

Suíte	Executados	Aprovados	Falhas	Resultado
-------	------------	-----------	--------	-----------

API	42	42	0	APROVADO
E2E Web	18	18	0	APROVADO
Mobile	12	12	0	APROVADO
TOTAL	72	72	0	APROVADO

Taxa total de sucesso: 100%. Resultado final do exemplo: PIPELINE APROVADO.

11. Evidências Produzidas

```

api-test-report/
  api-report.html
  api-junit.xml

e2e-playwright-report/
  index.html

e2e-failure-evidence/
  screenshots/
  videos/
  traces/

mobile-test-report/
  mobile-report.html

```

Esses arquivos ficam associados à execução do GitHub Actions e permitem verificar detalhadamente a causa de qualquer falha.

12. Benefícios da Solução

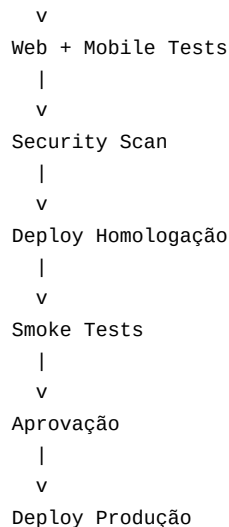
- Execução automática dos testes após alterações no código.
- Detecção antecipada de regressões.
- Padronização do processo de QA.
- Redução de validações manuais repetitivas.
- Histórico das execuções e das evidências.
- Maior segurança durante merges.
- Bloqueio automático de código defeituoso antes da produção.
- Feedback rápido para desenvolvedores.
- Identificação clara da camada em falha: API, Web ou Mobile.

13. Evolução Recomendada para Ambiente Profissional

```

Commit
|
v
Lint
|
v
Unit Tests
|
v
Build
|
v
API Tests
|

```



Ferramentas complementares possíveis: SonarQube para análise de qualidade, Allure Report para relatórios, Docker para padronização dos ambientes, Slack ou Microsoft Teams para notificações, Codecov para cobertura e Dependabot para análise de dependências.

Conclusão

A configuração proposta integra os testes de API, E2E Web e Mobile diretamente ao ciclo de desenvolvimento usando GitHub Actions. Cada alteração submetida ao repositório gera uma nova validação automática. Caso uma suíte crítica falhe, o pipeline também falha e pode impedir que a alteração seja incorporada à branch principal.

A solução oferece automação, rastreabilidade, geração de evidências, histórico de execuções e uma barreira automática contra regressões. A separação das suítes e a execução paralela contribuem para um bom equilíbrio entre cobertura e tempo de execução.